ShopEEZ

Full Stack MERN E-Commerce Platform

Solution Architecture

Phase: Project Design

Project: ShopEEZ | Date: June 2026 | Stack: MongoDB · Express · React · Node.js

1. System Architecture

ShopEEZ uses a decoupled client-server architecture following the MVC pattern.

React Frontend (Port 5173) !' [HTTP REST with JWT Bearer] !' Express Backend (Port 5000) !' [Mongoose ODM] !' MongoDB Atlas

2. Folder Structure

SHOPEEZZ/

client/src/

api/ — Axios instance with JWT interceptor

components/ — Navbar, Footer, ProductCard, ProtectedRoute

pages/ — Home, ProductList, ProductDetail, Cart, Checkout, Orders, Profile

pages/admin/ — Dashboard, Products, Orders, Users admin pages

redux/store.js — Redux store configuration

redux/slices/ — authSlice, cartSlice, productSlice, wishlistSlice

App.jsx — Route definitions (Public + Protected + Admin)

index.css — Global dark theme CSS custom properties

server/

config/db.js — MongoDB Atlas connection

controllers/ — authController, productController, cartController, orderController, adminController

middleware/ — authMiddleware (JWT), adminMiddleware (role), uploadMiddleware (Multer), errorHandler

models/ — User, Product, Cart, Order, Review

routes/ — authRoutes, productRoutes, cartRoutes, orderRoutes, adminRoutes

seed.js — Database seeder (products + admin account)

server.js — Express app entry point

3. Database Schema — Key Collections

Collection	Key Fields
User	name, email, password (bcrypt), role (user|admin), address, phone, avatar
Product	name, description, price, discountPrice, category, brand, stock, images[], isFeatured, tags[]
Order	user (ref), orderItems[], shippingAddress, paymentMethod, paymentStatus (pending|paid|failed), orderStatus (processing|shipped|delivered|cancelled), totalAmount
Cart	user (ref), items[], totalPrice
Review	user (ref), product (ref), rating, comment

4. Security Architecture

Layer	Mechanism	Implementation

Password Storage

Bcrypt (salt 10)

pre(save) hook in User.js

Authentication	JWT Bearer Token	protect middleware in authMiddleware.js
Authorization	Role-based (user/admin)	admin middleware in adminMiddleware.js
Admin Routes	Double middleware guard	router.use(protect); router.use(admin) in adminRoutes.js
CORS	Origin whitelist	Express CORS config in server.js